

Exact Cobordism Contraction and Succinct Block Cancellation

Toward quasi-polynomial unknot recognition in ProveIt

Research memorandum for the ProveIt project
Prepared with ChatGPT

7 October 2026

Abstract

We develop two complementary improvements for the exact Khovanov-scanning approach used in ProveIt's unknot recognizer. First, a compiled genus-zero cobordism composition factors through a square-free algebra on its connected components. We identify the exact quotient, prove its minimality for a fixed gluing plan, compute the rank of the linearized operation, and obtain a dense composition algorithm using $\text{poly}(b)2^b$ work instead of the current Cartesian monomial-pair loop, where $2b$ is the number of boundary endpoints. The arithmetic uses the classical fast subset-convolution algorithm, supplemented by characteristic-two polarization.

Second, we develop checked cancellation of matrix units and low-rank radical perturbations. For tensor-described interface maps, a block with 2^m generators can be cancelled without enumerating those generators. The running time depends polynomially on the tensor description and on the explicit interface dimensions, rather than on 2^m . This gives an unconditional quasi-polynomial algorithm for a precisely specified structured algebraic subproblem, not a general quasi-polynomial unknot recognizer.

The bundle includes a standard-library Python implementation, 29 passing test methods, source-audited adapters, paired measurements including regressions, and a verifiable 2^{40} -dimensional structured block example. Dense ten-variable compositions improve substantially in the measured harness, but the ordinary diagram tests never enter the dense path. We therefore recommend opt-in integration, not an unconditional replacement. Explicit tensor-rank obstructions and complete cost accounting identify the additional structural theorems needed for the global target.

Status. Research draft with complete proofs of the stated algebraic results and executable checks. No claim of a general quasi-polynomial recognition algorithm, formal Lean verification, exhaustive novelty search, or full-repository regression run is made.

Contents

1	Problem, contribution, and source audit	4
1.1	What must be improved	4
1.2	The default engine is not the older algebra adapter	4
1.3	Current mathematical context	5
1.4	What is being claimed as research progress	5
2	The coefficient algebra and compiled cobordisms	5
2.1	Representation and conventions	5
2.2	A plan and its local output map	6
3	An exact and minimal component quotient	6
4	Dense arithmetic, sparse polarization, and real bit costs	8
4.1	The dense multiplication algorithm	8
4.2	A characteristic-two sparsity improvement	9
4.3	Representation discipline	9
5	Matrix units: the scalar shortcut has a strict boundary	10
6	Schur cancellation through a low-rank interface	11
6.1	The categorical cancellation identity	11
6.2	Low-rank radical corrections	11
7	Succinct cancellation of exponentially large blocks	12
7.1	An explicit input model	12
7.2	Separable backgrounds	13
7.3	A fully specified 2^{40} -generator example	14
8	Why succinctness still needs structural theorems	14
8.1	An explicit product-tensor obstruction	14
8.2	A bounded-bond extension	15
8.3	Operations that do not establish the missing bound	15
9	End-to-end accounting and the remaining topological gap	16
9.1	What the arithmetic improvement proves for an explicit scan	16
9.2	A costed succinct schedule	16
9.3	Hierarchy costs: iteration counts are not running times	17
10	Implementation, validation, and measured performance	17

10.1 Implementation boundary	17
10.2 Validation actually performed	17
10.3 Benchmark protocol	18
11 Repository integration and reproducibility	20
11.1 Acceptance conditions for a default switch	20
12 Further research questions and concrete milestones	21
12.1 1. Where does dense coefficient arithmetic actually occur?	21
12.2 2. Can isotypic multiplicity blocks be extracted without expansion?	21
12.3 3. Which elementary operations preserve succinct descriptions?	21
12.4 4. Can heterogeneous matching interfaces be treated efficiently?	21
12.5 5. Which representation survives the equality-tensor obstruction?	21
12.6 6. Can one certify a nontrivial homology class before full expansion?	21
12.7 7. Should the three-braid route bypass scanning entirely?	22
12.8 8. Can extremal-degree computations supply useful early certificates?	22
12.9 9. Can hierarchy primitives meet an original-input size bound?	22
12.1010. Is there a logarithmic-depth decomposition with bounded interfaces?	22
12.1111. What should be formally verified first?	22
12.1212. Can the selector become predictably non-regressive?	23
13 Conclusion	23
A Claim ledger and artifact map	23
B Verifier specification and safety boundary	24

1 Problem, contribution, and source audit

1.1 What must be improved

The mathematical target is an exact decision procedure for an n -crossing knot diagram whose worst-case bit complexity is quasi-polynomial. We use “quasi-polynomial” for

$$\exp((\log(n+2))^{O(1)}),$$

and distinguish the stronger target $n^{O(\log n)} = \exp(O((\log n)^2))$ whenever the exponent matters. Neither a polynomial-time test of one invariant nor a fast implementation of one exponential backend establishes this target.

The inspected project is ProveIt, owned on GitHub by VladimirReshetnikov. The relevant directory is `Topology/UnknotRecognition`. Its current README explicitly separates the exact implementations from the announced quasi-polynomial goal [1]. The relevant Python implementation is a scanning computation in the dotted cobordism category, following Bar-Natan’s local construction [2]. It extends the partial complex at a crossing, deloops closed circles, and cancels invertible differential entries.

There are three different kinds of growth:

boundary complexity, morphism coefficient complexity, multiplicity of chain objects. (1)

They must not be conflated. The first part of this paper improves the second. The succinct-block part addresses the third on an explicit structured class. It does not assert that arbitrary knot diagrams belong to that class.

1.2 The default engine is not the older algebra adapter

The current `scan_fast.py` uses `planar.Planar`, with interned matching identifiers, compact object arrays, cached gluing plans, and a bucket/heap cancellation queue. The separately retained `algebra.BitAlgebra` is relevant to compatibility and ablation configurations. Its scalar involution shortcuts must not be mistaken for a description of the default composition routine.

The following source blobs were read through the repository connector on 7 October 2026. Blob identifiers are content identifiers, not an assertion that a complete checkout was fetched.

File within <code>fast/fastunknot/</code>	Source blob identifier
<code>algebra.py</code>	d6d28604ac39c29145aa52f91074539a5aeac949
<code>geometry.py</code>	96acc160dbf2df7981b2db98b602311712a01531
<code>planar.py</code>	1078526e7e7dbaf0b7267728105d870d94136769
<code>scan_fast.py</code>	2c1ad52d14296b109376af19668ae0eebb4c6fdd
<code>scan.py</code>	4acff3cb5ffb4c4ad226a8bf6dcda3d8dc0bb561

The bundled `algebra.py` fixture is byte-identical to its listed blob and its Git blob hash is checked. The `Planar/FastScan` execution fixtures were transcribed from the inspected source with shortened comments and formatting. They are *source-derived fixtures*, not byte-identical files from a complete checkout. Independent checks against the older set-based implementation help detect transcription mistakes; they do not replace an eventual full-checkout regression run.

1.3 Current mathematical context

Kelomäki and Schütz prove that the standard scanning and divide-and-conquer algorithms can take exponential time even for positive or alternating three-braids. Their obstruction is exponential total Betti number in a basis-expanding computation. They also give a polynomial algorithm for three-braid homology using a different description [4, Theorems 1.2–1.3]. Thus bounded boundary width is not enough to bound the number of explicit generators. This is a statement about those homology algorithms, not a lower bound for unknot decision, and not a lower bound for the repository’s entire filtered pipeline. In particular, an earlier filter may decide the same braid family cheaply.

Lackenby’s July 2026 preprint gives a hierarchy-based algorithm and bounds the number of visits to a step by $L(g+1)^L$, in terms of hierarchy length L and pattern complexity g . Section 9 explicitly leaves primitive running times insufficiently specified for a runtime estimate. The refined algorithm in Section 10 gives $L \leq 4c_q(H)$ [5, Propositions 9.1 and 10.2]. A linear bound in this geometric parameter is not automatically a logarithmic-depth or bit-complexity bound. Section 9 explains the precise extra accounting needed.

Unknot detection by reduced Khovanov homology is an established theorem of Kronheimer and Mrowka [6]. Our adapter does not change the detector, coefficients, grading convention, diagram validation, or preliminary filters. It replaces an algebra operation by an equal algebra operation.

1.4 What is being claimed as research progress

The deliverable is an exact, tested implementation of a component quotient and a structured cancellation mechanism, with proofs and counter-obstructions. Fast subset convolution, Schur complements, low-rank inverse identities, and product-tensor contraction are established algebraic ideas. We claim neither invention of these identities nor a literature-wide priority result. The contribution is their precise specialization, compatibility analysis, optimal-quotient characterization, succinct cancellation theorem, and measured integration into this research program.

2 The coefficient algebra and compiled cobordisms

2.1 Representation and conventions

For $s \geq 0$, let

$$R_s = \mathbb{F}_2[x_1, \dots, x_s] / (x_1^2, \dots, x_s^2). \quad (2)$$

For $S \subseteq [s]$, write $x_S = \prod_{i \in S} x_i$. Its coefficient vector has 2^s binary entries. The implementation encodes

$$F = \sum_{S \subseteq [s]} f_S x_S \quad \text{by} \quad \text{enc}(F) = \sum_{S \subseteq [s]} f_S 2^{\text{mask}(S)}.$$

An exponent in this integer encoding is a subset mask. Ordinary integer multiplication is *not* ring multiplication. Addition is XOR.

If a and b are crossingless matchings on the same boundary, the circles of $a \cup \bar{b}$ index a basis of dotted cobordisms in $\text{Hom}(a, b)$. We identify the underlying coefficient space with R_s . For endomorphisms of a matching with b arcs, composition is the multiplication of R_b . For different source and target matchings, the operation is specified by a gluing plan, rather than by multiplying coefficients in an arbitrarily chosen common ring.

We work in the ungraded coefficient algebra used for total-rank computation. Homological degrees of chain objects are still respected. Quantum shifts are not used to justify any cancellation. The matrix and tensor constructions later in the paper are restricted to copies of one matching, unless explicitly phrased in categorical terms.

2.2 A plan and its local output map

A genus-zero plan has connected components $j = 1, \dots, c$. Component j carries a disjoint set L_j of left input discs, a disjoint set Q_j of right input discs, a disjoint set B_j of output circles, and $e_j \geq 0$ fixed dots. The input sets partition their respective variable sets; the output sets partition the output variables. In the code these sets are bit masks.

The **BitAlgebra** plan stores (L_j, Q_j, B_j, e_j) . The default **Planar** additionally stores the precomputed list of masks $B_j \setminus \{i\}$. Our adapter removes only this redundant fifth field when compiling the new operation. It preserves the engine's own circle numbering.

For one component, put $z = z_j$ and define the linear map $\lambda_j : R_1 \rightarrow R_{B_j}$ by the following rule. On an input z^t , $t \in \{0, 1\}$, set $d = e_j + t$. If $d \geq 2$, the output is zero. Otherwise,

$$\lambda_j(z^t) = \begin{cases} 1, & B_j = \emptyset, d = 1, \\ 0, & B_j = \emptyset, d = 0, \\ x_{B_j}, & B_j \neq \emptyset, d = 1, \\ \sum_{i \in B_j} x_{B_j \setminus \{i\}}, & B_j \neq \emptyset, d = 0. \end{cases} \quad (3)$$

These are the multiplication/comultiplication/counit rules of the square-zero Frobenius algebra. A positive-genus component gives zero in this characteristic-two specialization: adding a handle acts by multiplication by $2x = 0$. Such a plan is represented by **None**.

3 An exact and minimal component quotient

Let a and d be the numbers of left and right input discs, and h the number of output circles. Define ring homomorphisms

$$\pi_L : R_a \longrightarrow R_c, \quad \pi_R : R_d \longrightarrow R_c,$$

by sending every input variable belonging to component j to z_j . Define

$$\Lambda = \lambda_1 \otimes \cdots \otimes \lambda_c : R_c \longrightarrow R_h,$$

using the disjoint output-variable blocks.

Theorem 3.1 (Component factorization). *For every valid compiled plan and every pair of coefficient vectors F, G , its bilinear cobordism operation is*

$$\mathcal{B}(F, G) = \Lambda(\pi_L(F)\pi_R(G)). \quad (4)$$

For a positive-genus zero plan, both sides are interpreted as zero.

Proof. It suffices to check input monomials. A monomial with two left dots in the same connected component maps to zero under π_L , because $z_j^2 = 0$. The original evaluator also kills it. The same

holds on the right. For the remaining monomials, multiplying their component images kills exactly those pairs with both a left and a right dot in one component. Every surviving component therefore receives zero or one variable dot. Adding its fixed dots and applying (3) is exactly the existing component evaluator. The output circles of different components are disjoint, so their outputs multiply by tensor product. Bilinearity completes the proof. $\square$

This factorization is not a heuristic aggregation by number of dots. It preserves which connected component receives each dot and all cancellations between projected monomials. For example, if x_1, x_2 belong to one component, then both $x_1 + x_2$ and $x_1 x_2$ project to zero, for different reasons. Forgetting the first cancellation would give an incorrect algorithm.

Proposition 3.2 (Exact rank of the linearized plan). *View $\mathcal{B}$ as the linear map $R_a \otimes R_d \rightarrow R_h$. Its rank is zero if a component has at least two fixed dots, or if a closed component has no inputs and no fixed dot. Otherwise its rank equals 2^u , where u is the number of components that have a nonempty output, at least one input, and no fixed dot.*

Proof. Regroup the input tensor factors by connected component. Each component with at least one input maps onto $\langle 1, z_j \rangle$ before fixed dots are imposed; a component with no inputs receives only 1. For an open component without a fixed dot and with an input, the two images in (3) are nonzero and have disjoint supports, hence are independent. This contributes rank two. Every other nonzero case has a one-dimensional image. The zero cases are exactly those stated. Ranks of tensor products of linear maps over a field multiply, which gives the formula. $\square$

Theorem 3.3 (Minimality for an ordinary nonzero composition plan). *Suppose every component has a left input, a right input, a nonempty output, and no fixed dot. Define $F \sim F'$ when*

$$\mathcal{B}(F, G) = \mathcal{B}(F', G) \quad \text{for every } G.$$

Then

$$F \sim F' \iff \pi_L(F) = \pi_L(F'). \tag{5}$$

If r_j is a chosen representative left variable in component j , the kernel of π_L is the ideal generated by $x_i + x_{r_j}$ for $i \in L_j \setminus \{r_j\}$. The quotient is R_c . A worst-case lossless encoding of the equivalence classes requires at least 2^c bits.

Proof. Equality after projection implies equivalence by Theorem 3.1. In the other direction, choose $G = 1$, so that $\pi_R(G) = 1$. The map Λ is injective: for each component its two images have disjoint nonempty supports, and this remains true for tensor products. Thus equality of $\mathcal{B}(F, 1)$ and $\mathcal{B}(F', 1)$ forces equality of their projections.

Quotienting by the stated ideal identifies the variables in each component and leaves exactly one square-zero variable per component. Hence the resulting quotient is R_c and its kernel is exactly that ideal. Surjectivity also follows by selecting one input variable from each component. There are 2^{2^c} different elements of R_c . An injective fixed-length binary representation therefore needs at least 2^c bits in the worst case; the same worst-case bound follows for unrestricted variable-length encodings by counting binary strings of smaller maximum length. $\square$

Remark 3.4 (Constructive recovery). Fix one output circle $i_j \in B_j$ in every component. From $\Lambda(H)$ one recovers the coefficient of z_S in H by reading the output monomial that dots all of B_j for $j \in S$ and all of $B_j \setminus \{i_j\}$ otherwise. No other component-occupancy pattern produces that monomial. This is an explicit left inverse, not merely a dimension argument.

For ordinary composition between matchings on $2b$ endpoints, a nonzero plan has the hypotheses of Theorem 3.3. Each connected component contains a gluing arc and consequently discs from both sides; surviving outer arcs supply output boundary. Moreover $a, d, h, c \leq b$. Thus the component quotient is the exact context-specific information that this composition needs. It need not be a sufficient representation for a *different* future gluing plan.

4 Dense arithmetic, sparse polarization, and real bit costs

4.1 The dense multiplication algorithm

In R_s , multiplication is disjoint subset convolution:

$$(FG)_S = \sum_{T \subseteq S} f_T g_{S \setminus T}. \quad (6)$$

We specialize the fast subset-convolution method of Björklund, Husfeldt, Kaski, and Koivisto [3]. For completeness, the algebraic correctness argument is included.

Lemma 4.1 (Ranked transforms). *All coefficients in (6) can be computed in $O((s+1)^2 2^s)$ field operations and $O((s+1)2^s)$ coefficient storage.*

Proof. Set $f_k(S) = f_S$ when $|S| = k$ and zero otherwise, and form ranked zeta transforms

$$\hat{f}_k(S) = \sum_{T \subseteq S} f_k(T), \quad \hat{g}_k(S) = \sum_{T \subseteq S} g_k(T).$$

Compute $\hat{h}_k(S) = \sum_{i=0}^k \hat{f}_i(S) \hat{g}_{k-i}(S)$, then invert the subset zeta transform in S . The resulting $h_k(S)$ is the sum of $f_A g_B$ over $A \cup B = S$ and $|A| + |B| = k$. Selecting $k = |S|$ forces $A \cap B = \emptyset$ and yields (6). In characteristic two the Möbius inversion uses the same XOR butterfly as the zeta transform. There are $O(s2^s)$ transform edges at each of $s+1$ ranks, and $O(s^2)$ scalar products per subset. $\square$

This classical result improves the usual 3^s disjoint-pair enumeration. The repository kernel instead loops through every pair of supported monomials before testing collisions, which can give $\Theta(4^s)$ pair visits on dense inputs. These are different comparison baselines.

Theorem 4.2 (Dense cobordism contraction). *Given the plan and dense input vectors, (4) can be evaluated using*

$$O\left((a+1)2^a + (d+1)2^d + (c+1)2^c + (h+1)2^h\right) \quad (7)$$

field operations and ordinary subset-mask operations. On matchings with b arcs this is $O((b+1)^2 2^b)$ in the corresponding word-RAM accounting, and $\text{poly}(b)2^b$ in a conservative bit-operation accounting.

Proof. For each input monomial, scan its set variables and record their components. Kill a monomial if two of its variables enter one component; otherwise XOR it into its projected coefficient. This costs the first two terms. Lemma 4.1 gives the product cost.

For the lift, closed components first require prescribed occupancy: a closed undotted component requires occupancy one, while a closed once-dotted component requires occupancy zero. An open once-dotted component also requires occupancy zero. In an open undotted component with r_j

output circles, the occupancy-zero and occupancy-one contributions together contain $r_j + 1$ distinct output monomials. Across components the total number of possible generated output monomials is at most

$$\prod_{B_j \neq \emptyset} (r_j + 1) \leq \prod_j 2^{r_j} = 2^h.$$

The supports associated with different admissible occupancy patterns are disjoint. Partial expansion work is bounded by the number of stages times the final output work. Occupancy checks are absorbed by the component-transform term. This proves (7). All subset masks have polynomially many bits in the displayed parameters, so moving from a word-RAM to bit operations adds only polynomial factors. $\square$

4.2 A characteristic-two sparsity improvement

Lemma 4.3 (Scalar square and polarization). *For $F \in R_s$, $F^2 = F(0)$. Consequently*

$$FG = F(F + G) + F(0) = G(F + G) + G(0). \quad (8)$$

A sparse product can therefore be evaluated using at most

$$\min\{|\text{supp } F| |\text{supp } G|, |\text{supp } F| |\text{supp}(F + G)|, |\text{supp } G| |\text{supp}(F + G)|\}$$

monomial-pair visits, plus representation traversal and a possible constant correction.

Proof. Every nonconstant square-free monomial squares to zero, and each cross term occurs twice. Squaring thus leaves only the constant coefficient. Expanding the two right sides of (8) proves the identities. Choose the representation with the fewest pairs and restore its indicated constant term. $\square$

The important place to apply this lemma is *after component projection*. Two originally different morphisms can have equal or nearly equal component images. A global coefficient comparison before projection misses this opportunity. This is not an identity between arbitrary morphisms before they are placed in a common component algebra.

4.3 Representation discipline

A Python integer holding 2^b coefficients is not a unit-cost machine word. Repeatedly removing one bit from a coefficient-sized integer may repeatedly copy a large object. The implementation reads its bytes once when traversing the support and repacks through a byte array. Ranked transforms use short, rank-indexed bit polynomials, not coefficient-sized integers as if each arithmetic operation were free.

The production prototype selects among a small-support original loop, projected sparse multiplication, and the ranked transform. Its default dense allocation limit is 18 component/output variables. Crossing that limit raises a resource exception on a selected factored path, not a knot verdict. The asymptotic theorem concerns the uncapped algorithmic family; to exercise it at parameter b , the limit must be at least the relevant variable counts. A fixed implementation cap cannot itself support an asymptotic claim beyond the cap.

5 Matrix units: the scalar shortcut has a strict boundary

Let $J = (x_1, \dots, x_b) \subset R_b$. Then $J^{b+1} = 0$. Write A_0 for the augmentation of a matrix, obtained by taking every entry's constant coefficient.

Theorem 5.1 (Unit blocks and logarithmic doubling). *A matrix $A \in \text{Mat}_M(R_b)$ is invertible if and only if $A_0 \in \text{Mat}_M(\mathbb{F}_2)$ is invertible. In that case, setting*

$$N = A_0^{-1}A - I \in \text{Mat}_M(J),$$

one has $N^{b+1} = 0$ and

$$A^{-1} = \left(\sum_{k=0}^b N^k \right) A_0^{-1}. \quad (9)$$

With $t = \lceil \log_2(b+1) \rceil$, the sum can be computed as

$$\sum_{k=0}^{2^t-1} N^k = \prod_{j=0}^{t-1} (I + N^{2^j}), \quad (10)$$

using $O(\log(b+2))$ matrix products. The exponent bound is sharp uniformly over matrix sizes.

Proof. An inverse descends to an inverse after augmentation. Conversely, if A_0 is invertible, every entry of a product of $b+1$ matrices in $\text{Mat}_M(J)$ is a sum of products of $b+1$ elements of J , so it vanishes. Thus $N^{b+1} = 0$. Multiplying the geometric sum by $I + N$ gives I on both sides. The factors in (10) are polynomials in the same matrix and hence commute with one another; expanding them chooses each exponent from 0 to $2^t - 1$ exactly once.

For sharpness, take the $(b+1)$ -by- $(b+1)$ strictly upper triangular matrix with $N_{i,i+1} = x_i$. Its b th power has top-right entry $x_1 \cdots x_b \neq 0$. Moreover $I + N$ has exact order $2^{\lceil \log_2(b+1) \rceil}$: every smaller power of two leaves a nonzero power of N . $\square$

Example 5.2 (A matrix unit is not generally an involution). Over $R_2 = \mathbb{F}_2[x, y]/(x^2, y^2)$,

$$A = \begin{pmatrix} 1 & x & 0 \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix}, \quad A^{-1} = \begin{pmatrix} 1 & x & xy \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix}.$$

The off-diagonal xy term cannot be omitted. The scalar identity $(1 + \nu)^{-1} = 1 + \nu$ does not extend to arbitrary matrix units.

With dense classical matrix multiplication and ring multiplication cost $\mu(b) = \text{poly}(b)2^b$, this inverse costs $O(M^3 \mu(b) \log(b+2))$, including a conservative allowance for augmentation inversion. This is not automatically faster than a well-chosen scalar cancellation sequence. Its purpose is to expose the smaller interface in the next section and to avoid an incorrect use of scalar involution in a future block implementation.

6 Schur cancellation through a low-rank interface

6.1 The categorical cancellation identity

Consider a differential block between consecutive homological degrees,

$$\partial = \begin{pmatrix} A & C \\ D & E \end{pmatrix} : U \oplus X \longrightarrow V \oplus Y, \quad (11)$$

where $A : U \rightarrow V$ is invertible.

Proposition 6.1 (Cancellation with explicit basis changes). *The block A is contractible and the remaining differential is*

$$S = E + DA^{-1}C. \quad (12)$$

The statement holds in an additive characteristic-two category, with all compositions well typed.

Proof. Multiply (11) on the left and right by the invertible block matrices

$$P = \begin{pmatrix} I & 0 \\ DA^{-1} & I \end{pmatrix}, \quad Q = \begin{pmatrix} I & A^{-1}C \\ 0 & I \end{pmatrix}.$$

Direct multiplication gives $P\partial Q = \text{diag}(A, S)$. Apply the same changes of basis to adjacent differentials. The equation $\partial^2 = 0$ and invertibility of A force the adjacent maps into and out of the A summand to vanish. The remaining two-term summand has contracting homotopy A^{-1} and can be removed. This also explains why a complete implementation must retain or reconstruct the basis changes, not merely the scalar dimensions of the removed summands. $\square$

6.2 Low-rank radical corrections

For the following matrix computations all objects are copies of one matching, so entries lie in the same R_b . Let

$$A = A_0 + UV, \quad B = A_0^{-1},$$

where A_0 now denotes an invertible background M -by- M matrix (not necessarily the augmentation), U is M -by- r , V is r -by- M , and $K = VBU$ has entries in J .

Theorem 6.2 (Interface Schur formula). *The matrix A is invertible and, writing $T = (I_r + K)^{-1}$,*

$$A^{-1} = B + BUTVB, \quad (13)$$

$$S = E + DBC + (DBU)T(VBC). \quad (14)$$

Thus the Schur update is determined by the four contractions

$$K = VBU, \quad L = DBU, \quad R = VBC, \quad H = E + DBC, \quad (15)$$

and equals $H + LTR$.

Proof. The matrix K is nilpotent by Theorem 5.1, so T exists. Multiplying the proposed inverse by A gives, after keeping all factors in their displayed order,

$$(A_0 + UV)(B + BUTVB) = I + U(I + T + KT)VB = I.$$

The product in the other order is also I , since $T(I + K) = I$. Substitution in (12) yields (14). No inverse of an expanded M -by- M matrix is needed once the four contractions are available. $\square$

This is a specialized low-rank inverse identity, not a new general matrix identity. The nontrivial algorithmic issue is whether the contractions can be formed without first expanding M objects. If U, V, C, D are ordinary explicit matrices, their input alone can cost $\Omega(Mr)$ or more. Hiding their construction behind an oracle would not establish a succinct algorithm. Section 7 gives an explicit representation in which this issue is resolved.

Proposition 6.3 (Coefficient-rank certificate). *Suppose a radical matrix has the expansion*

$$A - A_0 = \sum_{\emptyset \neq S \subseteq [b]} x_S H_S,$$

and each nonzero binary coefficient matrix has a factorization $H_S = P_S Q_S$ of inner dimension r_S . Then $A - A_0 = UV$ with U radical-valued and

$$r \leq \sum_{S: H_S \neq 0} r_S.$$

In particular, a dot-linear correction with every coefficient rank at most ρ has interface dimension at most $b\rho$.

Proof. Concatenate the blocks $x_S P_S$ horizontally to form U , and stack the Q_S vertically to form V . Their product is the displayed expansion. All weights x_S lie in J . $\square$

Finding these factorizations in a large explicitly represented coefficient matrix may itself be expensive. The proposition is a certificate construction, not a claim that low rank or its factorization is available for free.

7 Succinct cancellation of exponentially large blocks

7.1 An explicit input model

Let the hidden generator index be $\sigma \in \{0, 1\}^m$, so there are $M = 2^m$ generators. A product-tensor description of a vector $v \in R_b^M$ has the form

$$v(\sigma) = \sum_{t=1}^{s_v} \alpha_t \prod_{j=1}^m v_{t,j}(\sigma_j), \quad v_{t,j}(0), v_{t,j}(1), \alpha_t \in R_b. \quad (16)$$

It consists of actual coefficients, not an oracle for coordinates. Its size is $O(s_v(m+1)2^b)$ bits up to indexing and fixed-size metadata. A sufficient radical-valued certificate is that every α_t has zero constant coefficient.

Lemma 7.1 (Pairing without generator enumeration). *Given two descriptions (16), their bilinear pairing is*

$$\langle v, w \rangle = \sum_{t,u} \alpha_t \beta_u \prod_{j=1}^m (v_{t,j}(0)w_{u,j}(0) + v_{t,j}(1)w_{u,j}(1)). \quad (17)$$

It is computable in $O((m+1)s_v s_w)$ ring operations and does not enumerate 2^m coordinates.

Proof. Insert both descriptions into $\sum_{\sigma} v(\sigma)w(\sigma)$, exchange the finite sums, and factor the sum over independent binary choices σ_j . The ring is commutative, so factors belonging to the same mode can be grouped. This gives (17) and the stated operation count. $\square$

Theorem 7.2 (Succinct same-matching block cancellation). *Let $A = I_M + UV$ in (11), with $M = 2^m$, and suppose:*

- (i) *U has r columns, V has r rows, C has q columns, and D has p rows;*
- (ii) *each of these vectors has a product-tensor description with at most s terms;*
- (iii) *every term weight in every column of U is radical;*
- (iv) *$E \in \text{Mat}_{p \times q}(R_b)$ is explicit.*

Then the exact p -by- q Schur complement can be computed, without enumerating any of the M generator indices, in

$$O\left(\mu(b) \left[(m+1)s^2(r+p)(r+q) + r^3 \log(b+2) + pr^2 + prq + pq\right]\right) \quad (18)$$

operations in the same conservative ring/bit accounting as above. Here $\mu(b) = \text{poly}(b)2^b$ bounds one ring multiplication and includes addition cost.

Proof. The radical-weight certificate ensures every entry of U lies in J , so every entry of $K = VU$ lies in J . The four contractions in (15), with $B = I$, require respectively r^2, pr, rq, pq vector pairings. Their total number is $(r+p)(r+q)$. Lemma 7.1 gives the first term of (18). Compute the r -by- r inverse T using Theorem 5.1, then $H + LTR$ using dense rectangular products. These give the remaining terms. Every loop in the construction ranges over modes, tensor terms, coefficients, or explicit interface dimensions. No loop ranges over $\{0, 1\}^m$. $\square$

Corollary 7.3 (A precise quasi-polynomial subproblem). *For a family whose parameters satisfy*

$$b = O((\log(n+2))^2), \quad m, s, r, p, q \leq n^{O(1)},$$

the structured Schur problem of Theorem 7.2 is solvable in $n^{O(\log n)}$ bit time even when $M = 2^m$ is exponential in a polynomial of n .

Proof. All factors in (18) except $\mu(b)$ are polynomial in n and b . The latter is $2^{O((\log n)^2)}$ times a polynomial factor. Multiplying these bounds proves the claim. $\square$

This is unconditional for the stated algebraic input model. It is *not* an assertion that a knot diagram can be converted into such a sequence of blocks within these bounds. The latter would require a separate, topology-aware representation and closure theorem. Also, the explicit outer dimensions p, q are part of the bound; they must not secretly contain the same exponential multiplicity that was removed from the hidden index.

7.2 Separable backgrounds

Corollary 7.4 (An invertible tensor background). *Replace I_M by $A_0 = \bigotimes_{j=1}^m A_j$, where each $A_j \in \text{Mat}_2(R_b)$ is invertible. The same conclusion holds with an additional polynomial term for m small inverses and local transformations. The number of product-tensor terms does not increase.*

Proof. The inverse is $B = \bigotimes_j A_j^{-1}$. On a term in (16), applying B replaces the two-entry factor $v_{t,j}$ by $A_j^{-1}v_{t,j}$, without changing its scalar weight or term count. Transform the columns of U and C this way, leaving the rows of V, D unchanged. Form $VBU, DBU, VBC, E + DBC$ by the same pairings. Radical term weights remain radical. Theorem 6.2 completes the proof. $\square$

7.3 A fully specified 2^{40} -generator example

The supplied certificate uses $b = 2$, $m = 40$, $r = 3$, and $p = q = 1$. Write the first two index bits as a label $j \in \{0, 1, 2, 3\}$. For the remaining 38 modes, define u_j by fixing one parity class of modes to zero and leaving the other free; define v_j with the roles reversed. Both also fix their first two bits to the label j . Each vector has 2^{19} nonzero coordinates, but one product-tensor term. Their pairings are

$$\langle v_i, u_j \rangle = \delta_{ij}.$$

Set

$$U = (0, xu_0, yu_1), \quad V = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix}, \quad C = u_2, \quad D = v_0, \quad E = 0.$$

Then

$$K = VU = \begin{pmatrix} 0 & x & 0 \\ 0 & 0 & y \\ 0 & 0 & 0 \end{pmatrix}, \quad L = (0, x, 0), \quad R = (0, 0, 1)^T, \quad H = 0.$$

Consequently $S = xy \neq 0$. The xy contribution is exactly the matrix phenomenon of Example 5.2. Using the scalar involution shortcut on $I + K$ would return an incorrect answer.

The hidden module has 1 099 511 627 776 generators. The two nonzero outer-product rectangles in UV each have 2^{38} entries. The verifier reads the tensor description and recomputes the small interface; it neither creates these rectangles nor enumerates the generators. It reports an *algebraic* certificate, with `unknot_verdict: null`. This example demonstrates the exact representation theorem, not a trillion-crossing knot computation.

8 Why succinctness still needs structural theorems

8.1 An explicit product-tensor obstruction

Proposition 8.1 (Equality tensors need exponentially many product terms). *For $k \geq 1$, let $v(\sigma, \tau) = 1$ when $\sigma = \tau$ and zero otherwise, where $\sigma, \tau \in \{0, 1\}^k$. Any description of v as a sum of products over the $2k$ individual binary modes has at least 2^k terms, even if its coefficients lie in R_b .*

Proof. Apply augmentation to a proposed R_b description. Flatten the resulting tensor into a matrix with row index σ and column index τ . Each product term becomes a binary matrix of rank at most one. The target flattening is the 2^k -by- 2^k identity, of rank 2^k . Thus at least 2^k terms are necessary. $\square$

The function itself has a short factored Boolean formula,

$$v(\sigma, \tau) = \prod_{j=1}^k (1 + \sigma_j + \tau_j).$$

Therefore a short circuit does not imply a short description in the particular product-tensor format used here. The theorem is a representation obstruction; it does not claim that this tensor necessarily occurs in a difficult unknot instance.

There is a separate information-theoretic obstruction for arbitrary morphisms: R_b contains 2^{2^b} elements. A lossless worst-case encoding of all of them needs at least 2^b bits. The component quotient removes information irrelevant to one composition; it does not evade this count when an identity endomorphism context retains all coefficients.

8.2 A bounded-bond extension

The product-tensor format is not the only possible succinct format. For a fixed common mode ordering, consider a matrix-product description

$$v(\sigma) = A_1^{\sigma_1} \cdots A_m^{\sigma_m},$$

with end dimensions one and all intermediate dimensions at most χ ; coefficients lie in R_b . This subsection gives an algebraic extension, not an additional shipped backend.

Proposition 8.2 (Pairing bounded-bond descriptions). *Two such vectors with maximum bond dimension χ can be paired using $O(m\chi^3)$ ring operations. Substituting this pairing into Theorem 7.2 replaces its product-tensor pairing term by $O(m\chi^3(r+p)(r+q))$, provided radicality is supplied by a valid certificate.*

Proof. Start with the one-by-one matrix $Z_0 = (1)$. For the two families of cores $A_j^\varepsilon, B_j^\varepsilon$, update

$$Z_j = \sum_{\varepsilon \in \{0,1\}} (A_j^\varepsilon)^T Z_{j-1} B_j^\varepsilon.$$

Induction shows that Z_j sums the products of all partial paths with the same first j physical bits. At the last mode, Z_m is the desired scalar pairing. Each update uses a constant number of matrix products with dimensions at most χ . The Schur argument depends only on exact pairings and therefore applies unchanged. $\square$

For the equality tensor, interleaving paired modes gives a bounded-bond description despite its large product-tensor rank. However, low bond dimension for different incompatible mode orders does not by itself yield a cheap common-order pairing. A representation theorem must specify its ordering, intermediate ranks, and the costs of reordering and recompression.

8.3 Operations that do not establish the missing bound

A local tensor extension can preserve a short description for one step while multiplying its term count by a constant greater than one. Repeating that argument n times gives an exponential bound. Likewise, a recurrence $s_{i+1} \leq s_i^d$ with $d > 1$ is not harmless at logarithmic depth: it yields $s_0^{d^L}$, which can be far larger than quasi-polynomial. A useful global theorem needs a genuinely controlled description-size recurrence or a verified recompression step.

The code therefore never discards generators just because a rank or term-count cap has been reached. A cap raises a resource exception. Numerical tensor truncation, inferred equality from hashes without verification, and unproved identification of boundary objects are not sound replacements for chain maps.

9 End-to-end accounting and the remaining topological gap

9.1 What the arithmetic improvement proves for an explicit scan

Let B be the maximum number of matching arcs at any frontier. Let Q be the number of composition requests that actually reach the arithmetic backend, including requests not served by higher-level caches. Let H be the bit work for all remaining operations, including diagram processing, ordering, object and edge bookkeeping, transfers, and any rank computation.

Corollary 9.1 (A costed backend replacement). *With the dense contraction algorithm and uncapped resources, total runtime is bounded by*

$$H + Q \text{ poly}(B)2^B.$$

Thus $B = O((\log n)^2)$ and $H, Q \leq n^{O(\log n)}$ are sufficient for the strong quasi-polynomial target.

Proof. Apply Theorem 4.2 to each request and add the other work. Multiplication of two $n^{O(\log n)}$ quantities remains $n^{O(\log n)}$. $\square$

This statement deliberately displays H and Q . A bound on B alone does not bound the number of live objects, and therefore does not bound either quantity. Nor does a count of distinct matching *types* bound the number of copies of a type. Cached topology can be small while the differential between multiplicity spaces is large.

For a fully specified explicit scan with at most M live objects before each reduction, scalar eliminations number at most $O(nM)$ and a naive Schur update can touch $O(M^2)$ entries per cancellation. This gives a coarse $O(nM^3)$ bound on touched entry pairs, before charging their arithmetic and the other data structures. It is useful for conditional accounting, not a proof that M is small.

9.2 A costed succinct schedule

Theorem 9.2 (Composition of verified structured stages). *Suppose a recognition algorithm produces a sequence of at most n^a algebraic stages. Every stage is either a polynomial-size exact operation or a same-matching Schur problem of Theorem 7.2, with $b \leq C(\log(n+2))^2$ and all other displayed parameters at most n^a . Suppose the entire stage schedule, all required descriptions, all transitions, and the final decision certificate can be constructed and verified within $n^{O(\log n)}$ bit operations. Then the resulting recognition algorithm has bit complexity $n^{O(\log n)}$.*

Proof. Each structured stage has the asserted bound by Corollary 7.3. Summing over polynomially many stages preserves it. The stated construction, transition, and verification costs also have that bound. Correctness follows from the exact stage identities and the assumed sound final certificate. $\square$

The construction and transition hypotheses are substantial. This paper supplies neither a general schedule for knot diagrams nor a proof that crossing addition and cancellation maintain the required descriptions. The theorem records the exact missing interface between the algebraic result and a global recognizer, rather than treating it as implicit.

9.3 Hierarchy costs: iteration counts are not running times

Suppose a hierarchy routine has at most $O(L(g+1)^L)$ primitive-step visits and each visit costs at most $P(n)$ bit operations, measured against the original input size. Then

$$T(n) = O(P(n)L(g+1)^L). \quad (19)$$

For $L = O(\log n)$, $g = n^{O(1)}$, and $P(n) = n^{O(\log n)}$, this is $n^{O(\log n)}$. The elementary logarithm calculation is

$$\log T \leq O(1) + \log P + \log L + L \log(g+1) = O((\log n)^2).$$

This accounting applies to the step count discussed in Section 1, only after its additional hypotheses are proved.

Two common hidden losses deserve emphasis. First, $n^{O(\log n)}$ candidates at each of $O(\log n)$ recursively branching levels can give $\exp(O((\log n)^3))$, not the stronger $\exp(O((\log n)^2))$ target. Second, “polynomial in the current representation size” is insufficient when that size obeys $\ell_{i+1} \leq \ell_i^d$. Bounds must control every intermediate encoding in terms of the original n , including integer coordinate lengths, labels, provenance, and conversion costs.

The algebraic interface here may become a useful compression primitive, but it does not supply the layered-handle encoding, constructive surface routines, or topology-preserving cutting provenance required by the hierarchy route.

10 Implementation, validation, and measured performance

10.1 Implementation boundary

The package is standard-library Python. `subset.py` implements sparse and ranked multiplication; `kernel.py` compiles component quotients and lifts; `blocks.py` implements checked inverses and interfaces; `tensors.py` provides explicit product-tensor pairings and separable backgrounds. `adapters.py` creates a subclass of the caller’s `Planar` class without global monkey patches. The default crossing-transfer routine is unchanged. In particular, it retains its existing optimization that separates the locally touched circles from untouched renumbered circles.

The adapter must be installed before any crossing is processed. Otherwise interned matching identifiers, coefficient numbering, or caches may be incompatible. It clears its component-plan cache at every stage, preserves the existing shape cache, and passes the scanner’s resource-check callback into the new arithmetic. Scalar identities are used only when their source and target matching conditions hold.

The block/tensor code is a verified algebraic prototype, not yet a replacement cancellation driver for the complete scanner. It does not silently identify matrices between different matching types with matrices over one R_b .

10.2 Validation actually performed

The final test log reports **29 passing test methods**. The significant internal checks include 65,536 exhaustive products in R_3 , comparing the transform and polarized methods with unpolarized sparse multiplication; 2,245 monomial compositions across all noncrossing matching triples with at most three arcs, also compared against the older set-based cobordism oracle; all 2,744 four-arc matching

triples with seeded polynomial inputs; and 300 additional comparisons with the byte-identical `BitAlgebra` fixture.

Full raw scans were compared on six existing examples and 40 seeded valid braid closures. The checks compare the entire unnormalized homological-degree rank dictionary, not only a knot verdict. They also evaluate ∂^2 during the scans. The forced factored mode is tested in addition to the adaptive mode, since the latter may deliberately bypass the new kernel on sparse inputs.

Matrix tests verify inverses on both sides, the sharp nilpotence family, the failure of matrix involution, and agreement of small-interface and full Schur calculations. Tensor tests compare pairings against explicit expansion, compare succinct Schur complements against full matrices through 16 hidden generators, check a separable background, and reject tampered certificates. The 2^{40} example is verified through the proven contraction formula, not by an infeasible full expansion.

These are finite computational checks supporting the proofs. They are not a proof assistant certification or a claim that the entire current repository’s test suite was run. The original archive is retained only as an independent correctness oracle; it is not the timing baseline for the current-engine measurements.

10.3 Benchmark protocol

Measurements were made on CPython 3.13.5 under Linux x86-64. Each listed comparison uses five repetitions with fixed seeded inputs and alternating baseline/adaptive execution order. Garbage collection is performed before, not inside, the timed region. Every paired result is checked for equality. Raw timings and environment metadata are included in JSON and CSV. The execution environment is shared: CPU isolation and a stable clock frequency were not established. The ratios below describe these finite paired measurements, not hardware-independent speedup guarantees.

The microbenchmarks precompile the topological plan for both engines. “Cold” means that its monomial-result memo is empty; “warm” means a repeated call on the same inputs with this memo retained. This is not a benchmark of the scanner’s higher-level whole-composition cache, which could bypass both kernels. The reported speedup is the ratio of separate median times, not a confidence interval. Small microsecond-scale differences should not be overinterpreted.

Input	b	Memo	Baseline	Adaptive	Speedup
dense	4	cold	0.071	0.136	0.52
dense	6	cold	0.754	0.231	3.27
dense	8	cold	11.447	0.956	11.97
dense	10	cold	435.724	13.071	33.33
dense	6	warm	0.137	0.191	0.72
dense	8	warm	2.027	0.950	2.13
dense	10	warm	98.373	9.123	10.78
near equal	10	cold	182.002	0.861	211.47
near equal	10	warm	41.806	0.936	44.68
sparse	10	cold	0.025	0.028	0.87
sparse	10	warm	0.014	0.018	0.80

Table 1: Composition microbenchmarks. Times are milliseconds; speedup is baseline/adaptive. Topology is precompiled, whole-result caches are bypassed, and sparse regressions are included.

The dense ten-variable samples reduce the median cold composition time from 435.7 milliseconds to 13.1 milliseconds, a ratio of 33.3. The warmed baseline still visits every supported pair: its median

is 98.4 milliseconds, compared with 9.1 milliseconds for the adaptive kernel, a ratio of 10.8. Nearly equal samples benefit additionally from polarization after projection. Smaller or sparse cases can regress because the dispatch, checks, and quotient setup have a cost. These cases are retained in the table rather than excluded from the performance claim.

Diagram	Rank	Baseline	Adaptive	Ratio
Trefoil	6	0.604	0.586	1.032
Figure-eight	10	0.785	0.821	0.956
Eight-crossing unknot	2	1.377	1.543	0.893
Conway	66	5.852	5.590	1.047
Kinoshita–Terasaka	66	6.474	6.082	1.064
$T(3, 5)$	14	3.933	4.264	0.922
Forty-crossing braid unknot	2	9.462	8.434	1.122
Alternating three-braid (10)	242	6.022	6.097	0.988

Table 2: Raw scanning comparisons in the current-engine source-derived harness. Median milliseconds include the common order-selection routine; no preliminary recognition filters are used. All rank dictionaries agree.

Every adaptive scan in this table made *zero* factored-path calls at the threshold used. Where non-trivial compositions occurred, their support-product maximum was at most two. The measurements therefore provide no evidence of an end-to-end dense-kernel speedup on these ordinary diagrams. Variations in cases with no changed arithmetic are measurement noise or overhead, not algorithmic wins. The appropriate integration decision is to keep the new path opt-in and collect morphology profiles on larger workloads.

M	Full Schur	Build interface	Evaluate	Ratio
8	10.023	0.760	0.339	9.12
16	67.511	1.229	0.341	43.02
24	190.821	2.166	0.366	75.36

Table 3: Structured block experiment over R_3 , with supplied inner rank $r = 2$ and $p = q = 2$. Median milliseconds. Ratio compares the full Schur calculation with the sum of interface-building and evaluation medians.

These interface measurements use synthetic matrices satisfying the proved low-rank radical hypothesis, not matrices extracted from knot diagrams. The U, V factors are supplied to the interface method; the full A is supplied to the dense method. The “build” column includes forming all four contractions, so the comparison does not hide that work. It does not include discovering a low-rank factorization from an arbitrary matrix. Larger improvements in this structured setting must not be transferred to an unstructured scanner workload.

The last table measures certificate parsing into tensor objects, contractions, checked small inverse, and comparison with the claimed Schur result; file I/O and generation of the already supplied JSON object are outside the timed region. It illustrates description-sized work. It is not paired with a claimed measurement of an expanded trillion-dimensional matrix.

m	Hidden generators	Verify	Schur result
10	2^{10}	0.444	xy
20	2^{20}	0.482	xy
40	2^{40}	0.708	xy
80	2^{80}	1.208	xy
160	2^{160}	2.293	xy

Table 4: Description-sized verification of the explicit complementary-support family. Nine repetitions; median milliseconds. The represented matrix is never expanded.

11 Repository integration and reproducibility

The suggested destination is the additive directory

Topology/UnknotRecognition/research/frobenius-contraction/.

Nothing in the bundle requires overwriting the existing recognizer or changing its default result path. The intended initial integration is a research package and an opt-in adapter.

From the bundle root, run

```
PYTHONPATH=src:. python -m unittest discover -s tests -v
python benchmarks/run_benchmarks.py --repeats 5
python benchmarks/succinct_example.py
PYTHONPATH=src python -m unknot_frobenius.verify \
  examples/succinct_interface_2pow40.json
sh build.sh
```

Windows users can use `python run_tests.py` and the provided path setup in the benchmark scripts instead of setting `PYTHONPATH`. Building the PDF requires `pdflatex`; no bibliography processor is needed. The final `article.tex` is self-contained, with its tables and references embedded.

An actual-checkout validation script is included separately. It checks the two inspected production blob identifiers, imports the production `FastScan`, and compares unchanged versus adapted scans. Its presence is not a claim that it was run against a complete checkout during this session. Repository-wide Python and Rust regressions, cache behavior on larger inputs, and timing on the user’s target platform remain integration tasks.

11.1 Acceptance conditions for a default switch

A default switch should require: agreement on all existing invariants and degree ranks; a documented allocation/timeout policy; paired measurements showing no material regressions on ordinary inputs; actual invocation of the new path on representative expensive inputs; and a reproducible account of cache-warmness and coefficient supports. The present data satisfy arithmetic correctness checks but not the empirical case for a default switch.

For the tensor-block driver, a further acceptance condition is a typed certificate connecting the structured matrices to actual chain objects and differential maps. The existing algebraic verifier deliberately does not certify this topological provenance.

12 Further research questions and concrete milestones

12.1 1. Where does dense coefficient arithmetic actually occur?

Instrument the production Python and Rust engines to record input and projected support sizes, component counts, memo sizes, and repeated whole-result cache hits. The decisive dataset should include diagrams that survive the preliminary filters and require substantial exact-backend work. The milestone is a public, paired corpus demonstrating that the dense path is used often enough to offset dispatch overhead. A negative result is useful: it would redirect effort away from an already cheap kernel.

12.2 2. Can isotypic multiplicity blocks be extracted without expansion?

Define a typed representation for many copies of one matching and homological degree, including shifted objects where appropriate. Construct A_0, U, V, C, D, E together with proofs of their source and target types. The key bound is the cost of obtaining the factors from the scanner's state, not merely the cost after they have been supplied. An initial milestone is to identify a growing, diagram-derived family with polynomial-size factors and a measured multiplicity reduction.

12.3 3. Which elementary operations preserve succinct descriptions?

Give explicit formulas for crossing addition, delooping, basis changes, and the Schur update in the chosen tensor or bounded-bond representation. Track coefficient width, term count or bond dimension, explicit outer dimensions, and construction time simultaneously. The desired theorem controls these parameters by the original diagram size throughout the computation; an exponential recurrence disguised as repeated local simplification does not suffice.

12.4 4. Can heterogeneous matching interfaces be treated efficiently?

The tensor implementation currently uses one coefficient ring R_b for a same-matching sector. A general scanner has $\text{Hom}(a, b)$ bimodules between different matchings. Extend the four-contraction formula using their correctly typed compositions, and use the component quotient to contract coefficients without imposing a false common-ring structure. A successful extension needs both an algebraic proof and an explicit cost bound for basis changes and gluing plans.

12.5 5. Which representation survives the equality-tensor obstruction?

Implement the bounded-bond pairing of Proposition 8.2, then compare it with product-tensor descriptions on diagram-derived interfaces. The equality tensor is a mandatory regression example. Study common-order width, not just the separate bond dimensions of two inputs. An ordering/re-compression algorithm needs exact certificates; numerical low-rank truncation is not sufficient for a decision procedure.

12.6 6. Can one certify a nontrivial homology class before full expansion?

Design cycle/cocycle or chain-retraction certificates that prove the rank is larger than the unknot rank while retaining short descriptions. For example, several cycles together with dual cocycles and

a full-rank pairing would provide an independently checkable lower bound. The missing work is to construct and evaluate those maps cheaply in the tangle representation. A partial-complex object count alone is not such a certificate, since later attachments may create cancellations.

12.7 7. Should the three-braid route bypass scanning entirely?

The polynomial three-braid result cited in Section 1 suggests a specialized exact backend rather than further optimization of a basis-expanding computation on that class. Implement its normal-form and invariant formulas with explicit coefficient and bit-length checks. Compare all relevant ranks against the current backend on a broad finite range. This is an implementation of a known result, not a new theorem being claimed here, but could be a high-value performance improvement.

12.8 8. Can extremal-degree computations supply useful early certificates?

Analyze when a small collection of exactly computed homological degrees suffices to rule out the unknot, and how to choose those degrees adaptively. The specification must state which part of the homology is certified and what it implies for decision. It must also give uniform parameter dependence before allowing the number of retained degrees to grow with input size.

12.9 9. Can hierarchy primitives meet an original-input size bound?

For the geometric route, specify the binary representation of layered handles, boundary patterns, surfaces, and cutting provenance. Implement one primitive with a worst-case cost and an independently checkable output certificate. The milestone is not a fast demonstration alone, but a bound on every integer and data structure in terms of the original crossing number. Only then can a step-count estimate be converted to runtime using (19).

12.10 10. Is there a logarithmic-depth decomposition with bounded interfaces?

Search for a decomposition invariant that simultaneously controls frontier size, description size, and recursion depth. Prove an explicit recurrence and solve it without suppressing the dependence of an inner algorithm on a growing parameter. Both the $n^{O(\log n)}$ and the broader quasi-polynomial target should be tracked. A counterfamily forcing large interfaces in a proposed representation would be a substantive negative result, not merely a failed implementation.

12.11 11. What should be formally verified first?

The most self-contained targets are the component factorization, exact plan-rank formula, scalar polarization, and nilpotent matrix inverse. Next formalize the typed block-cancellation identity and tensor pairing. These are small algebraic interfaces with clean specifications and extensive finite tests. A full proof-carrying recognizer must ultimately also verify diagram parsing, topology-changing moves, and the connection to the final detection theorem.

12.12 12. Can the selector become predictably non-regressive?

The current threshold is a heuristic. Develop a selector based on component count, projected support, and memo availability, or run competing exact kernels under a shared budget when the overhead is justified. A theorem about arithmetic work does not automatically control wall-clock overhead. Preserve a deterministic result, explicit resource exceptions, and paired benchmark evidence for each added selection rule.

13 Conclusion

The component algebra is an exact, minimal sufficient quotient for a fixed nonzero gluing plan. It gives a provable dense arithmetic improvement and a safe place to exploit characteristic-two polarization. The block analysis identifies precisely where scalar involution stops being valid, and the tensor-interface theorem demonstrates that exponential multiplicity can sometimes be removed without expanding it. The supplied verifier realizes this distinction on a 2^{40} -generator structured block.

The empirical result is deliberately narrower than the algebraic result: dense compositions improve substantially in the tested harness, but the ordinary scanned diagrams remain sparse and show no established overall speedup. The global quasi-polynomial target now has a sharper set of obligations: construct succinct diagram-derived blocks, preserve their descriptions under exact operations, and charge all transitions and topology certificates to the original input size. Those obligations are research problems stated here, not assumptions silently converted into a recognition theorem.

A Claim ledger and artifact map

Claim	Status and limitation	Artifact
Component factorization	Proved for every valid compiled plan; exact current-operation specialization.	<code>kernel.py</code> ; Theorem 3.1
Minimal quotient and rank	Proved; minimality needs the stated open, undotted, input-covering hypotheses.	Theorem 3.3; <code>linearized_rank</code>
Dense speed bound	Proved algorithmic bound; ranked convolution is classical; allocation caps are separate.	<code>subset.py</code> ; Theorem 4.2
Block inverse and Schur	Proved and checked; scalar involution is not used on matrix units.	<code>blocks.py</code>
Succinct cancellation	Proved for supplied tensor-described same-matching blocks. No general construction from a diagram.	<code>tensors.py</code> ; Theorem 7.2
Bounded-bond extension	Proved pairing algorithm; not implemented in this bundle.	Proposition 8.2
Measured speedups	Finite paired measurements with raw data and negative cases. No universal wall-clock claim.	<code>results/</code>

Claim	Status and limitation	Artifact
Production adapter	Implemented and exercised with source-derived current-engine fixtures. Complete-checkout run remains required.	<code>adapters.py</code> ; <code>integration/</code>
General quasi-polynomial unknot recognition	Not established by this work.	Section 9 states missing hypotheses

B Verifier specification and safety boundary

The JSON schema `unknot-frobenius-interface-v1` specifies the ring variable count, number of binary modes, product-tensor terms for the columns of U, C and rows of V, D , the explicit matrix E , and a claimed Schur complement. A term consists of a ring coefficient and one two-entry factor per mode. Coefficients use the bit encoding in Section 2.

The verifier validates coefficient ranges, matrix dimensions, common mode counts, and radical weights in U . It constructs the four contractions itself, computes and checks the small inverse, and compares the resulting Schur matrix with the claim. It imposes explicit input-size and parameter limits. It does not treat successful algebraic verification as proof that the input was extracted from a knot complex. In particular, neither a JSON description nor a large represented dimension certifies a topological result.

A future topological certificate must additionally bind the algebra to a diagram, matching types, homological degrees, previous chain maps, and the exact representation transitions. A final knot verdict also needs the existing detector’s hypotheses. This separation prevents a mathematically correct matrix computation from being misreported as an unknot certificate.

References

- [1] Vladimir Reshetnikov and ProveIt contributors. *ProveIt: Topology/UnknotRecognition*, repository source inspected 7 October 2026.
<https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>. Relevant content identifiers are recorded in Section 1 and `reference/provenance.json`.
- [2] Dror Bar-Natan. Fast Khovanov homology computations. *Journal of Knot Theory and Its Ramifications* **16**(3) (2007), 243–255. doi:10.1142/S0218216507005294. Preprint [arXiv:math/0606318](https://arxiv.org/abs/math/0606318).
- [3] Andreas Björklund, Thore Husfeldt, Petteri Kaski, and Mikko Koivisto. Fourier meets Möbius: fast subset convolution. In *Proceedings of STOC 2007*, 67–74. doi:10.1145/1250790.1250801. Preprint [arXiv:cs/0611101](https://arxiv.org/abs/cs/0611101).
- [4] Tuomas Kelomäki and Dirk Schütz. *On computational complexity of Khovanov homology*. Preprint, 5 January 2026, 30 pages. [arXiv:2601.02119v1](https://arxiv.org/abs/2601.02119v1).
- [5] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. Preprint, 25 July 2026, 54 pages. [arXiv:2607.23350v1](https://arxiv.org/abs/2607.23350v1). Author’s PDF:
<https://people.maths.ox.ac.uk/lackenby/algorithm-incompressible-250726.pdf>.
- [6] Peter B. Kronheimer and Tomasz S. Mrowka. Khovanov homology is an unknot-detector. *Publications Mathématiques de l’IHÉS* **113** (2011), 97–208. doi:10.1007/s10240-010-0030-y. Preprint [arXiv:1005.4346](https://arxiv.org/abs/1005.4346).